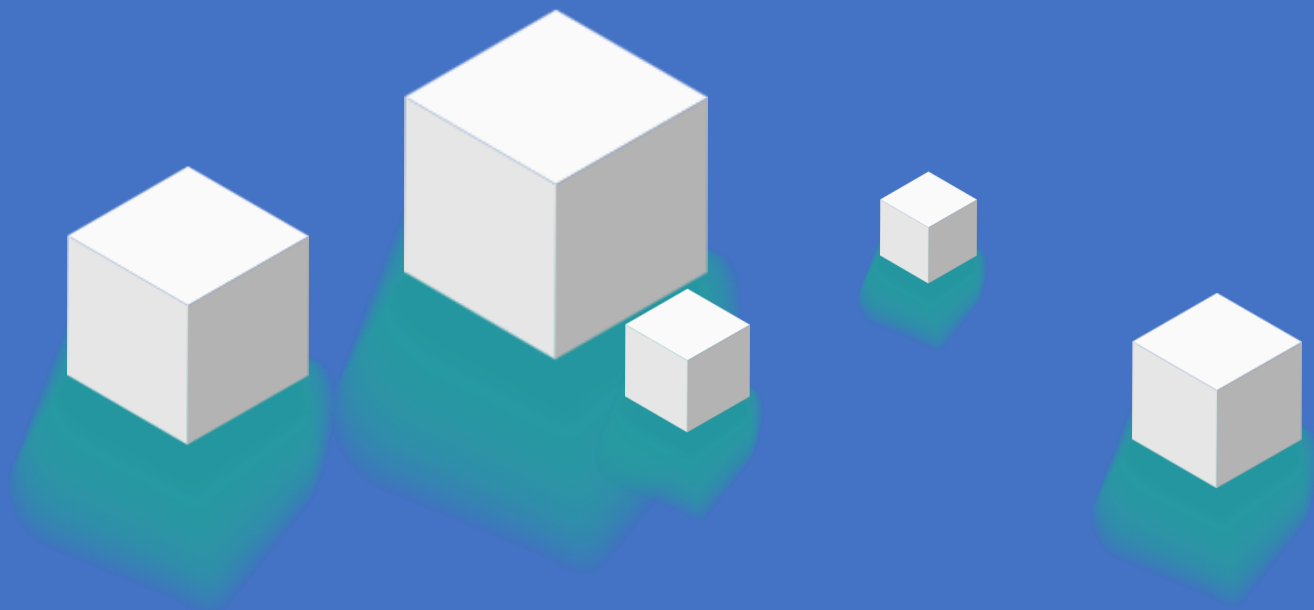


论文复现与策略进化



>> 今天的学习目标

论文复现与策略进化

- 论文复现与策略进化

论文复现五步法

复现Checklist

- MASTER论文解读

论文要解决的问题

数据长什么样

创新点1: 双重注意力Transformer

创新点2: 市场引导特征选择(Gate)

源码解读

- CASE: MASTER数据与因子

- CASE: MASTER截面收益预测

数据预处理对比

- CASE: XGBoost截面收益预测

MASTER 与 XGBoost的对比

- 前沿展望

论文复现与策略进化

为什么要学论文复现?

Thinking: 拿到一篇量化论文, 从哪里开始复现?

量化领域的 信息差往往藏在学术论文里。顶会论文经过严格同行评审, 方法可靠度远高于公众号文章。

但大多数人拿到论文后会陷入两个极端:

- 只看结论不看方法 → 知其然不知其所以然
- 试图一字不差地复现 → 被细节淹没, 迟迟无法推进

五步法提供了一条系统化的复现路径

论文复现五步法

Step1: 提取核心假设

每篇论文都在验证一个假设。先找到它, 再决定是否值得复现。

读论文时问自己三个问题:

1. 作者试图解决什么问题?
2. 核心假设是什么? (如果这个假设不成立, 论文还有价值吗?)
3. 这个假设在中国A股市场是否合理?

=> 以MASTER为例:

- **问题:** 如何更准确地预测股票收益率?
- **假设:** 市场整体状态能引导模型动态选择重要因子, 提升预测准确率
- **A股验证:** 合理 — A股牛熊转换剧烈, 不同市场环境下有效因子确实不同

论文复现五步法

Step2: 理解数学公式

找到论文的核心公式, 逐项理解每个符号的含义。不要跳过公式!

MASTER的核心前向传播公式:

$$\hat{y} = \text{Decoder}(\text{TemporalAttn}(\text{SAttn}(\text{TAttn}(\text{Gate}(x, m) * x))))$$

拆解:

x : (N, T, 158) 158维Alpha因子

m : (N, 63) 63维市场信息

$\text{Gate}(x, m)$: (N, 158) 市场引导的因子权重

$\text{Gate} * x$: (N, T, 158) 加权后的因子

TAttn : (N, T, D) 时间注意力 (同一股票, 不同天)

SAttn : (N, T, D) 截面注意力 (同一天, 不同股票)

TempAttn : (N, D) 时间聚合 (T步 $\rightarrow$ 1个向量)

Decoder : (N,) 线性解码 $\rightarrow$ 预测收益率

论文复现五步法

Step3: 梳理算法步骤

将公式转化为可执行的伪代码或流程图:

输入 (N, T=8, F=222)

- └- 分离: 前158维 = Alpha因子, 第158~221维 = 市场信息, 第222维 = 标签
- └- Gate: 63维市场信息 $\rightarrow$ Linear $\rightarrow$ Softmax $\rightarrow$ 158维权重
- └- 加权: Alpha因子 $\times$ Gate权重 $\rightarrow$ 特征选择后的因子
- └- 投影: Linear(158, 256) $\rightarrow$ 映射到隐空间
- └- 位置编码: PositionalEncoding $\rightarrow$ 加上时间步位置信息
- └- T-Attention: 同一股票内部, 8个时间步之间的自注意力
- └- S-Attention: 同一时刻, N只股票之间的自注意力
- └- TemporalAttention: 8个时间步聚合为1个向量
- └- 解码: Linear(256, 1) $\rightarrow$ 输出预测收益率

论文复现五步法

Step4: 设计回测方案

与论文一致的实验设置是复现成功的关键:

MASTER论文的实验设置:

数据集: CSI300 / CSI800

训练集: 2016-01-01 ~ 2018-12-31

验证集: 2019-01-01 ~ 2019-12-31

测试集: 2020-01-01 ~ 2020-12-31

回看窗口: $T = 8$ 天

学习率: $1e-5$

Epochs: 40

Dropout: 0.5

Beta(CSI300): 5.0

论文复现五步法

Step5: 对比结果验证

对比论文报告的指标, 允许合理误差范围(5-10%):

复现结果 vs 论文报告:

IC: 0.075 vs 0.083 (误差 9.6%, 在合理范围内 ✓)

ICIR: 0.52 vs 0.58 (误差 10.3%, 临界值 △)

如果误差>15%, 需要排查:

- 数据源是否一致? (Qlib vs Wind vs Tushare)
- 预处理参数是否对齐? (clip范围, 标准化方法)
- 随机种子是否固定?

复现Checklist

在复现任何论文之前, 逐项确认以下四类问题:

1. 数据一致性

- 时间范围是否与论文一致? (训练/验证/测试的划分)
- 数据频率是否一致? (日频/分钟频/Tick)
- 标的范围是否一致? (CSI300/CSI500/全A)
- 数据源是否可靠? (Wind/Qlib/Tushare)

复现Checklist

Thinking: 什么是Qlib?

微软研究院亚洲团队于 2020 年 9 月开源的 AI 量化投资平台, <https://github.com/microsoft/qlib>

39K星标, 利用 AI 赋能量化研究, 覆盖从探索想法到生产落地的完整流程。

- 多范式建模: 支持监督学习、市场动态建模、强化学习三种范式
- 完整流程: 涵盖数据处理 → 因子挖掘 → 模型训练 → 回测 → 组合优化 → 订单执行
- 模型丰富: 内置 40+ 种 SOTA 模型, 包括 LightGBM、LSTM、Transformer、GATs、TRA 等
- 高性能数据层: 自研二进制格式, 比 MySQL 快 50 倍以上 (7.4 秒 vs 365 秒)
- 自动化研发: 集成 RD-Agent (2024 年发布), 基于大模型实现自动因子挖掘和模型优化

复现Checklist

2. 参数明确性

- 论文的默认超参数是否都找到了?
- 学习率/batch_size/epoch/隐藏层维度等
- 是否有未在正文中提及但在附录或代码中的参数?
- 温度参数/dropout/正则化系数等细节参数

复现Checklist

3.评价指标对齐

指标	定义	有效阈值
IC	预测值与真实收益的Pearson相关系数	> 0.05
ICIR	IC均值 / IC标准差，衡量信号稳定性	> 0.3
RankIC	预测排名与真实排名的Spearman秩相关	> 0.08
RankICIR	RankIC均值 / RankIC标准差	> 0.5
夏普比率	风险调整后收益	> 1.0
最大回撤	最大净值回撤幅度	< 20%

复现Checklist

4. 预处理一致性

- 标准化方法: Z-Score / RobustZScore / MinMax / Rank
- 去极值方法: 3sigma截断 / MAD / 分位数截断
- 缺失值处理: 前值填充 / 均值填充 / 0填充 / 删除
- 是否做了行业/市值中性化?

CASE: 均线交叉策略

CASE: 均线交叉策略

TO DO: 用一个最简单的 论文复现 热身 — 均线交叉策略:

```
class MAStrategy(bt.Strategy):
    """
    假设的论文描述: MA5上穿MA20买入, 下穿卖出
    标的: 贵州茅台(600519.SH), 回测期: 2024~2025
    """

    params = (('ma_short', 5), ('ma_long', 20))

    def __init__(self):
        self.ma_short = bt.indicators.SimpleMovingAverage(
            self.data.close, period=self.params.ma_short
        )
        self.ma_long = bt.indicators.SimpleMovingAverage(
            self.data.close, period=self.params.ma_long
        )
        self.crossover = bt.indicators.CrossOver(self.ma_short, self.ma_long)
```

```
def next(self):
    if not self.position:
        if self.crossover > 0: # 金叉 → 买入
            self.buy()
    else:
        if self.crossover < 0: # 死叉 → 卖出
            self.sell()
```


CASE: 均线交叉策略

运行结果

均线交叉策略(MA5/MA20) | 600519.SH | 2024-01-02 ~ 2025-12-31 | 481个交易日

总收益: -31.64% | 年化: -18.07% | 最大回撤: 31.64% | 夏普: -2.12 | 卡玛: -0.57

交易: 17次 | 胜率: 29.4% | 盈亏比: 0.26 | 利润因子: 0.12 | 最大连亏: 5次

CASE: 均线交叉策略

Thinking: 简单均线交叉在茅台上表现为负收益, 这说明什么?

在茅台这种大波动股票上, 频繁的均线金叉死叉会产生大量虚假信号, 导致 两边挨打。

这正是为什么我们需要更智能的方法 — 比如MASTER的Gate机制, 能根据市场状态动态选择因子。

=> 论文通常涉及复杂的架构

MASTER论文解读

MASTER论文解读

MASTER: Market-Guided Stock Transformer for Stock Price Forecasting

发表: AAI-2024 (人工智能顶级会议)

机构: 上海交通大学 DMTai 实验室

任务: 给定N只股票过去T天的F维特征, 预测未来收益率

GitHub: <https://github.com/SJTU-DMTai/MASTER>



顶会论文、代码开源

金融+DL结合: Transformer架构 + 金融领域知识(Gate机制)

效果显著: 在CSI300/CSI800数据集上超越多个基线

CSI300 (沪深300指数)

CSI300 (沪深300指数)

全称：沪深300指数 (CSI 300 Index)

指数代码：000300 (沪市) 、 399300 (深市)

发布机构：中证指数有限公司 (由上交所和深交所共同出资设立)

发布时间：2005年4月8日

基日：2004年12月31日，基点1000点

成分股数量：300只

选样方法：沪深300指数由沪深市场中规模大、流动性好、最具代表性的300只证券组成，反映A股市场大盘股的整体表现。成分股每半年调整一次（1月和7月初实施），调整比例不超过10%

CSI800 (中证800指数)

CSI800 (中证800指数)

全称：中证800指数 (CSI 800 Index)

指数代码：000906 (沪市)、399906 (深市)

发布日期：2007年1月15日

基日：2004年12月31日，基点1000点

成分股数量：800只

中证800 = 沪深300 + 中证500

这800只股票综合反映了中国A股市场大中小盘上市公司的整体表现，覆盖了约67%的A股市值

论文要解决的问题

Thinking: 论文想解决什么问题?

给定 N只股票 过去 8天的 158个量价因子, 预测每只股票未来5天的收益率排名。

假设你是一个基金经理, 管理着沪深300的300只股票。每天早上你需要决定: 今天重仓买哪几只?

传统做法的问题:

问题1: 只看今天的快照

XGBoost等模型只看今天的52个指标(RSI、MACD等), 完全忽略了"这些指标过去8天是怎么变化的"。

比如: RSI今天是65, 但它是从30涨上来的(强势), 还是从90跌下来的(转弱)? 单日快照看不出来。

论文要解决的问题

问题2: 每只股票独立分析

XGBoost把每只股票当作独立的样本, 完全忽略股票之间的联动。

比如: 宁德时代涨停了, 锂电池板块的其他股票大概率也会跟涨。这种"板块联动"传统模型看不到。

问题3: 所有市场环境用同一套模型

牛市和熊市, 你看的指标应该不一样。但XGBoost对所有因子的权重是固定的。

比如: 牛市看动量(追涨), 熊市看波动率(避险), 震荡市看布林带(高抛低吸)。

论文要解决的问题

MASTER的三个创新, 对应这三个问题:

问题1: 只看今天, 忽略时序变化

=> MASTER方案: 看过去8天, 用注意力机制找出哪天最重要

=> 对应模块 TAttention (时间注意力)

问题2: 每只股票独立, 忽略联动

=> MASTER方案: 300只股票一起看, 找出谁和谁有关联

=> 对应模块 SAttention (截面注意力)

问题3: 因子权重固定, 不适应市场

=> MASTER方案: 先看大盘状态, 再决定今天重点看什么指标

=> 对应模块 Gate (市场引导门控)

数据长什么样

核心任务: 股票价格预测

输入张量 X : $(N, T, F) = (300/800, 8, 222)$

输出向量 y : $(N,)$ = 每只股票的预测收益率

N = 股票数量

CSI300 约 300只 (沪深300成分股)

CSI800 约 800只 (中证800成分股)

T = 回看窗口 = 8天

使用过去8个交易日的数据进行预测

F = 特征维度 = 222维

158个Alpha因子 + 63维市场信息 + 1个标签 = 222维



Thinking: 为什么不是 $(1, F)$ 而是 (N, T, F) ?

传统方法(如XGBoost): 每只股票独立建模, 输入是 $(1, F)$ 的一维向量

MASTER: 所有股票一起建模, 输入是 (N, T, F) 的三维张量

- T 维度: 捕捉时序依赖 — 过去8天的演变趋势
- N 维度: 捕捉截面相关性 — 不同股票之间的联动

数据长什么样

2026年3月23日这一天, MASTER看到的数据: 300层 x 8行 x 222列 的数据立方体

	第0维: KBAR因子	第1维: 收益率	...	第157维: WVMA	第158维: 沪深300收益	...	第220维: 中证800成交波动	第221维: 标签
贵州茅台, 3月13日	0.32	-0.15	...	0.87	-0.42	...	1.23	NaN
贵州茅台, 3月14日	0.28	0.21	...	0.91	0.15	...	0.98	NaN
...	...							
贵州茅台, 3月23日	-0.11	0.45	...	1.02	0.83	...	0.76	0.52

这里只是贵州茅台这一只股票的8天数据 (一个 8x222 的矩阵)。

沪深300有300只股票, 所以一共有 300个这样的矩阵, 叠在一起就是 (300, 8, 222)。

Alpha158

Thinking: 158个Alpha158因子是什么?

这是微软Qlib平台定义的标准量价因子集, 全部从 OHLCV (开高低收量) 计算得来

类别	维度	典型因子	说明
KBAR/KMID/KLEN	22	(Close-Open)/Open	k线的"身材" -- 胖瘦、高矮
KUP/KLOW	12	上下影线长度	多空博弈的痕迹 -- 影线越长, 博弈越激烈
OPEN/CLOSE/HIGH/LOW	24	价格偏离度	当前价格相对历史的位置
VWAP	6	成交均价偏离	机构的真实买入成本
VOL/VSTD	12	成交量均值/波动	市场活跃度
CORD/CORR	12	价量相关系数	量价配合度 -- 放量涨 vs 缩量涨
CNTP/CNTN/CNTD	18	涨跌天数统计	类似RSI的思路
SUMP/SUMN/SUMD	18	涨跌幅度统计	上涨力度 vs 下跌力度
RESI/BETA	12	回归alpha/beta	个股相对大盘的独立波动
VMA/VSTD2/WVMA	22	均线类	趋势方向和强度

每个因子在 5/10/20/30/60 日 多个窗口下计算, 所以虽然基础类型不多, 但排列组合后有158维。

63维市场信息

Thinking: 63维市场信息是什么?

63维 = 3个指数 x 21维统计量

指数	每个指数的21维
沪深300 (CSI300) 中证500 (CSI500) 中证800 (CSI800)	1维: 当日收益率 20维: 在5/10/20/30/60日窗口下, 分别计算: - 收益率均值、收益率标准差 - 成交额均值、成交额标准差 (5个窗口 x 4个统计量 = 20维)

这63维描述的是大盘现在是什么状态：是牛市(收益率高、波动小)? 还是熊市(收益率低、波动大)?

63维市场信息

Thinking: 标签: 预测什么?

标签是未来5天的归一化收益率

```
# 原始收益率
```

```
r_raw = (第5天收盘价 - 第1天收盘价) / 第1天收盘价
```

```
# 截面Z-Score归一化 (同一天所有股票一起标准化)
```

```
r_label = (r_raw - 当日所有股票收益率均值) / 当日所有股票收益率标准差
```

```
# 去极端值: 排名前2.5%和后2.5%的标签被剔除
```

```
# 这样做是为了防止少数暴涨暴跌的股票主导训练
```

MASTER预测的不是 这只股票会涨几个点, 而是 在300只股票中, 这只股票的排名会排第几。

=> 评估指标用 RankIC(排名相关性) 而不是 MAE(预测误差)。

=> 做投资本质上是排序: 买预期排名靠前的, 卖预期排名靠后的。

创新点1: 双重注意力Transformer

创新点1: 双重注意力Transformer

输入(N,T,F) → Gate特征选择 → 线性投影 → 位置编码

→ TAttention → SAttention → TemporalAttention

→ 线性解码 → 输出(N,)

创新点1: 双重注意力Transformer

TAttention — 时间注意力 (Intra-stock)

作用: 对同一只股票的不同时间步做自注意力

输入: (N, T, D) 每只股票 T 个时间步, 每步 D 维

注意力方向: 沿 T 维度计算

意义: 股票A的第1天关注第3天? 第5天关注第8天?

捕捉时序依赖, 如"放量后缩量"的模式

类比: 相当于NLP中Transformer对句子内部词的注意力

SAttention — 截面注意力 (Inter-stock)

作用: 对同一时刻的不同股票做自注意力

输入: (N, T, D) 在 T 的每个时间步, 对 N 只股票做注意力

注意力方向: 沿 N 维度计算

意义: 茅台关注五粮液? 宁德关注比亚迪?

捕捉截面相关性, 如行业/风格联动

类比: 把每只股票当作一个token, 做"股票序列"的注意力

创新点1: 双重注意力Transformer

TemporalAttention — 时序聚合

作用: 将 T 个时间步聚合为1个向量

方法: 用最后一天($t=T$)的表示作为query

对所有时间步计算注意力权重, 加权求和

输出: (N, D) 每只股票一个 D 维表示

直觉: 用最近一天的状态 去查询 历史每一天有多重要

自适应地决定应该更关注近期还是远期的信息

TAttention: 同一只股票内部, 哪天最重要?

AI直觉: 复盘一只股票的过去8天

假设你在分析贵州茅台过去8天的走势:

天	1月8日	1月9日	1月10日	1月11日	1月12日	1月13日	1月14日	1月15日
涨跌	-0.5%	-0.3%	+0.2%	+5.1%	+1.2%	+0.8%	+0.3%	+0.1%
成交量	正常	正常	正常	放量3倍	放量1.5倍	正常	缩量	缩量

人眼一看就知道: 1月11日发生了重要事件(暴涨+放量), 应该重点关注这一天。

=> TAttention做的就是同样的事: 让模型自动发现8天中哪天的信息最重要。

TAttention: 同一只股票内部, 哪天最重要?

自注意力机制: 三步走

Step1, 为每天生成 Query / Key / Value

输入: 贵州茅台 8天的256维表示, 形状 (8, 256)

通过三个不同的线性变换, 生成 Q/K/V

$Q = W_q * X$ # (8, 256) -- Query: "我想找什么?"

$K = W_k * X$ # (8, 256) -- Key: "我有什么?"

$V = W_v * X$ # (8, 256) -- Value: "我的内容是什么?"

直觉:

Q: "第5天问: 过去8天中, 哪天的放量突破信号跟我最相关?"

K: "每天回答: 我这天有什么特征可以匹配你的查询"

V: "匹配上之后, 把我这天的实际信息传给你"

TAttention: 同一只股票内部, 哪天最重要?

Step2, 计算注意力权重

```
# 每天和每天之间算相似度 (点积)
attention_scores = Q * K^T  # (8, 8) 的矩阵

#   第1天 第2天 第3天 第4天 第5天 第6天 第7天 第8天
# 第1天 [ 0.3  0.1  0.0  0.8  0.2  0.1  0.0  0.0 ]
# 第2天 [ 0.1  0.2  0.1  0.7  0.3  0.1  0.1  0.0 ]
# ...
# 第8天 [ 0.0  0.0  0.1  0.9  0.2  0.1  0.0  0.1 ]
#
#           ^^^
# 所有天都觉得第4天(暴涨那天)最重要!

# 逐行softmax归一化
attention_weights = softmax(attention_scores, dim=-1)
```

Attention: 同一只股票内部, 哪天最重要?

Step3, 加权聚合

用注意力权重对 Value 做加权求和

```
output = attention_weights * V  # (8, 256)
```

结果: 每天的表示都"吸收"了其他天的重要信息

比如第8天的输出 = $0.05 * V1 + 0.02 * V2 + \dots + 0.45 * V4 + \dots + 0.10 * V8$

^ ^ ^ ^

第4天(暴涨日)贡献了最多信息

TAttention: 同一只股票内部, 哪天最重要?

多头注意力: 同时关注不同方面

TAttention使用 N1=4个头, 每个头的维度 = $256/4 = 64$ 维:

头	可能学到的模式
Head 1 (64维)	价格趋势: 关注哪几天价格连续上涨
Head 2 (64维)	成交量变化: 关注放量/缩量的转折点
Head 3 (64维)	波动率变化: 关注波动加剧的日子
Head 4 (64维)	近期 vs 远期: 关注最近几天 vs 更早的信息

4个头的输出拼接回 256维, 经过 FFN(前馈网络) + 残差连接 + LayerNorm。

SAttention: 不同股票之间, 谁和谁有关联?

AI直觉: 板块联动效应

在同一个时间点, 不同股票之间存在联动关系:

比如: 2026年某天的板块联动

事件	直接影响	间接影响 (SAttention能捕捉)
宁德时代涨停	宁德时代自身	亿纬锂能、天赐材料 (锂电池板块跟涨)
美联储加息	银行股(招行、平安)反应最快	地产股、消费股延迟反应 (2-3天后)
原材料涨价	上游资源股先涨	下游制造股后跌 (成本压力传导)

SAttention: 不同股票之间, 谁和谁有关联?

Thinking: SAttention怎么做?

在每个时间步, 把300只股票排成一行, 做自注意力:

```
# 在时间步 t=4 (第4天), 所有300只股票的表示:
H_t4 = [h_茅台_t4, h_平安_t4, h_宁德_t4, ..., h_比亚迪_t4] # (300, 256)

# 生成 Q/K/V
Q = W_q * H_t4 # (300, 256)
K = W_k * H_t4 # (300, 256)
V = W_v * H_t4 # (300, 256)

# 计算注意力 (这次有缩放因子!)
scores = Q * K^T / sqrt(d/H) # (300, 300) -- 每对股票之间的相似度
weights = softmax(scores)
```

```
# 注意力矩阵 (300, 300) 长这样:
#      茅台 平安 宁德 美的 比亚迪 ...
# 茅台 [ 0.15 0.02 0.01 0.01 0.01 ... ] <- 茅台关注自己+五粮液
# 平安 [ 0.01 0.18 0.01 0.01 0.01 ... ] <- 平安关注自己+招行
# 宁德 [ 0.01 0.01 0.12 0.02 0.20 ... ] <- 宁德关注比亚迪!
# ...
```


TemporalAttention: 把8天压缩成1个判断

Thinking: 经过TAttention和SAttention后, 每只股票有8个时间步的表示, 但我们只需要1个预测值

怎么把 (8, 256) 压缩成 (256,)?

方案对比

方案	做法	缺点
方案A: 取最后一天	直接用第8天的表示	浪费了前7天的信息
方案B: 平均池化	8天取平均	所有天等权, 不分轻重
方案C: TemporalAttention (MASTER采用)	用最后一天作为query, 自适应加权	最优 -- 自动决定每天权重

TemporalAttention: 把8天压缩成1个判断

TemporalAttention的数学

```
# 输入: 一只股票8天的表示 Z = (8, 256)
# 用最后一天(第8天)作为 query
query = Z[7]          # (256,) -- "站在今天, 回头看过去"

# 计算每天与今天的相关度
scores = Z * W * query # (8,) -- 每天一个分数
weights = softmax(scores) # (8,) -- 归一化为权重

# 假设权重: [0.02, 0.03, 0.05, 0.25, 0.15, 0.12, 0.18, 0.20]
#               ^^^^          ^^^^
#               暴涨那天权重高    最近一天权重也高

# 加权求和
e = sum(weights * Z)    # (256,) -- 这只股票的最终表示
```

TemporalAttention的直觉

用"今天的状态"作为查询条件, 去回顾过去8天, 自适应地决定应该更重视近期还是远期。

如果最近几天发生了重大变化(比如暴涨), 模型会给近期更高的权重。

如果最近很平淡但一周前有个关键事件, 模型会给一周前更高的权重。

Summary

TAttention vs SAttention 的区别

维度	TAttention	SAttention
注意力方向	沿 T 维度（时间步之间）	沿 N 维度（股票之间）
输入维度	(B, N, T, d) → 关注 T=8 个时间步	(B, N, T, d) → 关注 N=300/800 只股票
缩放因子	无（不除以 $\sqrt{d}$ ）	有（除以 $\sqrt{d/H}$ ）
转置操作	无	有（transpose 使 N 维可做注意力）
设计原因	T=8 很小，不需要防止注意力过于集中	N=300/800 很大，需缩放避免梯度消失
核心功能	捕捉时序依赖（单只股票的历史趋势）	捕捉截面关联（股票间的市场结构）
类比	Transformer 的时序自注意力	图注意力网络（GAT）的节点间注意力

Summary

Thinking: 为什么需要双重注意力?

传统Transformer只在一个维度做注意力:

- GPT/BERT: 只在序列(时间)维度
- ViT: 只在空间(patch)维度

MASTER同时在两个维度做注意力:

- 时间维度(TAttention): 一只股票内部的时序关系
- 截面维度(SAttention): 不同股票之间的关联

=> 这种设计特别适合金融数据的 面板数据 (panel data)结构。

创新点2: 市场引导特征选择(Gate)

Thinking: 牛市和熊市, 你会看同样的指标吗?

不同市场环境下, 不同因子的重要性不同:

- 牛市: 动量因子(ROCP, RSI)更重要 → 趋势跟随
- 熊市: 防御因子(波动率, 换手率)更重要 → 风险规避
- 震荡市: 均值回归因子(乖离率)更重要 → 高抛低吸

传统方法: 对所有因子等权使用, 或人工选择固定的因子子集

MASTER: 根据当前市场状态, 动态调整每个因子的权重

创新点2: 市场引导特征选择(Gate)

63维市场信息的构成

```
# 伪代码 (摘自MASTER论文)
m = []
for S in [CSI300, CSI500, CSI800]:    # 3个指数
    m += [market_index(S, -1)]        # 当日收益率 (1个)
    for d in [5, 10, 20, 30, 60]:    # 5个时间窗口
        m += [mean(S.return, d)]     # 收益率均值
        m += [std(S.return, d)]      # 收益率标准差
        m += [mean(S.amount, d)]     # 成交额均值
        m += [std(S.amount, d)]      # 成交额标准差

# 每个指数:  $1 + 5 \times 4 = 21$  维
# 总计:  $3 \times 21 = 63$  维
```

创新点2: 市场引导特征选择(Gate)

Gate机制的数学描述

输入: gate_input = 63维市场信息向量 (最后一天的市场状态)

Gate网络:

$$w = \text{softmax}(\text{Linear}(\text{gate_input}) / \beta)$$

其中:

Linear: 63维 $\rightarrow$ 158维的全连接层

β : 温度参数, 控制Gate的"锐度"

softmax: 归一化为概率分布, 确保权重非负且和为1

应用Gate:

$$x_{\text{gated}} = x * (F' \times w)$$

其中:

x : 原始158维因子 (N, T, 158)

w : Gate权重 (N, 158)

F' : 缩放因子 = 158, 使期望值等于原始值

创新点2: 市场引导特征选择(Gate)

温度参数beta的作用

beta值	效果	类比
0.1 (低温)	接近 one-hot, 只保留 1-2 个因子	硬选择 — 信息损失大但噪声少
1.0 (默认)	较为集中, 保留 5-10 个主要因子	信息保留和噪声过滤的平衡
5.0 (CSI300)	适度分散, 论文 CSI300 最优值	因子池较小时适当分散
10.0 (高温)	接近均匀分布, 所有因子近似等权	退化为不做特征选择

创新点2: 市场引导特征选择(Gate)

Gate的直觉理解

想象你是一个基金经理, 每天早上做投资决策:

1. 先看一眼大盘指数 – 目前沪深300涨了? 跌了? 波动大吗? 成交量活跃吗? (= 63维市场信息)
2. 根据大盘状态, 决定今天重点看哪些指标 (= Gate输出的权重)
3. 牛市: 今天大盘涨, 成交量放大 => 重点看动量因子 (追涨)
4. 熊市: 今天大盘跌, 波动加大 => 重点看波动率因子 (避险)
5. 震荡市: 大盘横盘, 量能萎缩 => 重点看均值回归因子 (高抛低吸)

=> Gate机制就是用神经网络自动学习这个 看指标 的过程。

源码解读

Gate类

```
class Gate(nn.Module):
    def __init__(self, d_input, d_output, beta=1.0):
        self.trans = nn.Linear(d_input, d_output)
        # 线性变换: 63维市场信息 → 158维因子权重空间
        self.d_output = d_output # 保存输出维度, 用于缩放
        self.t = beta           # 温度参数

    def forward(self, gate_input):
        output = self.trans(gate_input)
        # gate_input: (N, 63) → output: (N, 158)
        output = torch.softmax(output / self.t, dim=-1)
        # 除以温度, 再softmax归一化
        return self.d_output * output
        # 乘以158缩放: softmax输出和为1, 平均1/158
        # 乘以158后平均值回到1, 不改变原始特征量级
```

```
# 输入: 最后一天的市场信息 m = (N, 63)
# 线性层: 把63维映射到158维(因子个数)
z = W * m + b # z的形状: (N, 158)

# W是 (158, 63) 的权重矩阵, 通过训练学到
# 直觉: 每个因子对应一个 评分规则, 根据63维市场状态给因子打分

# 除以温度beta, 再做softmax归一化
w = softmax(z / beta) # w的形状: (N, 158), 每个值在0~1之间, 所有值加起来=1
# beta=5 (CSI300), beta=2 (CSI800)

# 乘以158(因子个数), 让平均权重回到1.0
gate_weight = 158 * w # 平均值=1.0, 重要因子>1, 不重要因子<1
# 应用到因子上: 逐元素相乘
x_gated = x * gate_weight # (N, 8, 158) * (N, 1, 158) = (N, 8, 158)
```

源码解读

Thinking: 假设只有5个因子, 看Gate怎么工作?

今天大盘状态: 沪深300连涨3天, 成交量放大 (牛市信号)

因子	线性层输出 z	z / beta (beta=5)	softmax(z/beta)	x 5 (缩放)	含义
动量因子	3.2	0.64	0.32	1.60	放大60% (牛市追动量)
RSI	2.1	0.42	0.26	1.28	放大28%
波动率	0.5	0.10	0.18	0.92	基本不变
换手率	-0.8	-0.16	0.14	0.69	缩小31%
乖离率	-1.5	-0.30	0.10	0.51	缩小49% (牛市不需要均值回归)

在牛市环境下, Gate自动放大了动量和RSI因子, 缩小了均值回归因子。

如果明天大盘暴跌(熊市信号), 这些权重会自动调整 -- 波动率和换手率会被放大, 动量被缩小。

源码解读

温度参数beta的设置:

beta很小 (如 $\beta=0.5$) -- 果断型

softmax输出接近one-hot: [0.90, 0.05, 0.02, 0.02, 0.01]

只保留1-2个因子, 其他几乎被关掉

风险: 信息损失大, 但噪声也少

beta很大 (如 $\beta=10$) -- 犹豫型

softmax输出接近均匀: [0.21, 0.21, 0.20, 0.19, 0.19]

所有因子近似等权, Gate相当于不起作用

退化为传统方法: 不做特征选择

论文发现: CSI300用 $\beta=5$ (因子质量高, 不需要太激进地筛选),

CSI800用 $\beta=2$ (股票多、噪声大, 需要更强的特征选择)。

源码解读

MASTER主类

```
class MASTER(nn.Module):
    def __init__(self, d_feat, d_model, t_nhead, s_nhead,
                  gate_input_start_index, gate_input_end_index, beta, ...):
        self.feature_gate = Gate(63, d_feat, beta)
        self.layers = nn.Sequential(
            nn.Linear(d_feat, d_model),    # 特征投影: 158 → 256
            PositionalEncoding(d_model),   # 位置编码: 给时间步加位置信息
            TAttention(d_model, t_nhead),   # 时间注意力
            SAttention(d_model, s_nhead),   # 截面注意力
            TemporalAttention(d_model),     # 时序聚合
            nn.Linear(d_model, 1)          # 解码器: 256 → 1
        )
    def forward(self, x):
        src = x[:, :, :self.gate_input_start_index]
        # 取出Alpha因子: (N, T, 158)
```

```
        gate_input = x[:, -1, start:end]
        # 取出最后一天的市场信息: (N, 63)

        src = src * torch.unsqueeze(self.feature_gate(gate_input), dim=1)
        # Gate权重: (N, 158) → unsqueeze → (N, 1, 158)
        # 与因子相乘: (N, T, 158) * (N, 1, 158) = (N, T, 158)
        # 广播: 同一只股票在所有时间步使用相同的Gate权重

        output = self.layers(src).squeeze(-1)
        # 经过 Linear→PE→TAttn→SAttn→TempAttn→Linear
        # (N, T, 158) → ... → (N, 1) → squeeze → (N,)
        return output
```

MASTER 与 XGBoost的对比

维度	XGBoost	MASTER Transformer
模型类型	梯度提升决策树（截面）	Transformer（时序+截面）
输入格式	50+特征（单日快照）	222特征×8天（多日序列）
输入形状	(N, F) = (N, 50+)	(N, T, F) = (N, 8, 222)
特征选择	人工选择 + SHAP分析	Gate机制自动动态选择
跨股票建模	独立建模（无交互）	SAttention建模相关性
时序建模	无（单日快照）	TAttention + TemporalAttn
训练方式	滚动窗口 / CPU几分钟	GPU批训练 / 数小时
可解释性	高（SHAP特征重要性）	中（注意力权重可视化）
部署难度	低（sklearn/xgb, CPU即可）	高（PyTorch, GPU>=8GB）
适用场景	中低频 / 因子挖掘	日频 / 端到端预测
过拟合风险	较低（树模型天然正则化）	较高（需要Dropout等正则化）
数据量要求	中等数据量即可	需要大量数据

MASTER 与 XGBoost的对比

1. 特征工程 vs 端到端学习

XGBoost路线: 人工构造因子 → SHAP特征选择 → 模型训练 (三步走)

MASTER路线: 原始因子 → Gate自动选择 → 注意力建模 → 预测 (端到端)

2. 独立建模 vs 联合建模

XGBoost把每只股票当作独立样本, 忽略股票间的关联。

MASTER通过SAttention捕捉股票间的联动效应:

比如: 宁德时代涨停 → 锂电池板块共振, S-Attention能捕捉这种跨股票的信号传递。

3. 静态特征 vs 动态特征

XGBoost: 所有市场环境使用相同的因子权重

MASTER: Gate机制根据市场环境动态调整因子权重

MASTER 与 XGBoost的对比

4. AI量化建议

小资金/个人投资者: XGBoost更实用

→ 数据获取容易, 训练快, 解释性好, 容易调试

机构/大资金: MASTER类模型更有优势

→ 数据和算力充足, IC每提升0.01都有收益价值

=> 最佳实践: 组合使用

→ XGBoost做基准模型, Transformer做增强

→ 两者预测结果加权融合, 取长补短

MASTER的因子体系

158个Alpha158因子 (Qlib标准)

全部基于量价数据(OHLCV)计算, 每个因子在5/10/20/30/60日多窗口下计算:

类别	名称	维度	说明
KBAR	K线形态因子	10	(Close-Open)/Open 等
KLEN	K线长度因子	6	(High-Low)/Open 等
KMID	K线中间值因子	6	K线中点位置
KUP/KLOW	上下影线因子	12	影线长度特征
OPEN/CLOSE	开收盘偏离	12	收益率类特征
HIGH/LOW	最高最低价偏离	12	极值偏离特征
VWAP	成交均价偏离	6	VWAP/Close-1
VOL/VSTD	成交量因子	12	量的均值与波动
CORD/CORR	价量相关性	12	Pearson/Spearman相关系数
CNTP/CNTN/CNTD	涨跌统计	18	类RSI指标
SUMP/SUMN/SUMD	涨跌幅度	18	涨跌强度
RESI/BETA	回归特征	12	个股alpha和beta
VMA/VSTD2	均线与波动	12	技术分析类
WVMA	加权均线	6	量价加权波动
合计		~158	

MASTER的因子体系

158个Alpha158因子 (Qlib标准)

全部基于量价数据(OHLCV)计算, 每个因子在5/10/20/30/60日多窗口下计算:

维度	MASTER (Alpha158)	50+因子
因子总数	158维个股 + 63维市场 = 221维	50+维 (含行业/基本面)
数据源	纯量价数据 (OHLCV)	量价 + 财务 + 行业
K线形态因子	34维 (非常详细)	少量 (影线比等)
回归特征	RESI/BETA (12维)	无
价量相关性	CORD/CORR (12维)	无
市场状态	63维Gate输入 (3指数×21)	无 (不建模市场整体)
行业因子	无	申万一级行业哑变量
基本面因子	无	PE/PB/ROE/营收增长等

MASTER用纯量价因子(221维)做到了比L11含基本面因子(50+维)更高的IC
=> 因子数量和时序建模能力可以弥补信息源的不足

Summary

TAttention 里的 T 表示 Time（时间）：

在同一只股票内部，对过去 $T=8$ 个时间步做自注意力，刻画“时序上的演变”。

SAttention 里的 S 表示 Stock（股票 / 截面）：

在同一天、同一时刻，在 N 只股票之间做自注意力，刻画“谁和谁一起动、龙头带动、板块联动”等截面关系。

$T = \text{Temporal}$ （时间维）， $S = \text{Stock}$ （股票维 / 截面维）。

也有的资料也会说成“spatial over stocks”，但 MASTER 里更贴切的说法就是 Stock 维度的注意力。

CASE: MASTER数据与因子

CASE: MASTER数据与因子

TO DO: 从MySQL加载真实A股数据(10只代表性大盘股), 完成建模前的关键准备工作

Step1, 加载10只A股, 计算52维因子 => 构建分析用面板数据

Step2, 因子分布诊断(偏度/峰度/异常值) => 了解原始数据的 健康状况

Step3, RobustZScoreNorm vs MAD+Z-Score对比 => 理解MASTER与L11的预处理差异

Step4, 因子相关性分析 => 发现冗余因子, 指导特征选择

Step5, MASTER 63维市场信息分析 => 对比MASTER的数据规模

=> 建模前的必要工作: 了解因子分布、处理异常值、发现冗余因子。

CASE: MASTER数据与因子

运行结果:

数据: 10只A股, 7,230行 x 52因子, 耗时0.5s

因子分布诊断:

异常值率最高: new_low_20d(12.75%), macd_adx_cross(12.16%)

高偏度因子($|\text{skew}| > 1$): 28/52

高峰度因子($\text{kurtosis} > 5$): 25/52

--> 原始因子普遍存在厚尾分布, 必须预处理

预处理对比 (以rsi_14为例):

原始范围: [15.66, 93.69]

RobustZScore: [-2.74, 3.00] 偏度=0.013

MAD+Z-Score: [-2.86, 3.69] 偏度=0.021

clip到[-3,3]被裁比例: 0.10%

因子相关性:

高相关对($|r| > 0.8$): 66对

Top: ret_5d ~ momentum_5d ($r=1.000$), macd_hist ~
macd_adx_cross ($r=0.971$)

树模型对共线性不敏感, 但神经网络需要去冗余

MASTER CSV:

3665天 x 63维, 范围[-3, 3](已RobustZScoreNorm处理)

CASE: MASTER数据与因子

关键发现:

- 原始因子分布不理想

52个因子中过半存在高偏度或厚尾, 直接喂给模型会影响训练稳定性

- RobustZScoreNorm更安全

输出严格限制在 $[-3, 3]$, 适合深度学习(梯度稳定);

MAD+Z-Score范围不固定, 适合树模型(不依赖数值范围)

- 因子冗余率较高

66对高相关因子($|r| > 0.8$), 说明52维中存在信息重叠, 但XGBoost能自动处理这种冗余

数据预处理对比

MASTER: RobustZScoreNorm

```
# MASTER的预处理流程
median = np.median(x)          # 中位数
mad = np.median(np.abs(x - median))  # 中位数绝对偏差(MAD)
robust_std = mad * 1.4826      # MAD到标准差的转换系数

x_normalized = (x - median) / robust_std # 标准化
x_clipped = np.clip(x_normalized, -3, 3) # 裁剪到[-3, 3]
```

L11: MAD去极值 + Z-Score

```
# L11课程的预处理流程
median = np.median(x)
mad = np.median(np.abs(x - median))

upper = median + 5 * mad      # 上界
lower = median - 5 * mad      # 下界
x_clipped = np.clip(x, lower, upper) # 裁剪

x_normalized = (x_clipped - mean) / std # Z-Score标准化
```


数据预处理对比

预处理对比总结

	MASTER (RobustZScoreNorm)	L11 (MAD去极值 + Z-Score)
第1步	用中位数+MAD做标准化	用中位数+MAD做去极值
第2步	硬裁剪到 [-3, 3]	用均值+标准差做标准化
中心化基准	中位数 (鲁棒)	均值 (受极值影响)
离散度基准	MAD x 1.4826 (鲁棒)	标准差 std (受极值影响)
输出范围	严格 [-3, 3]	不固定

两者核心一致: 去极值 + 标准化 + 截面处理, 区别在于鲁棒性参数的选择。

数据预处理对比

假设某天 RSI 因子在 10 只股票上的原始值:

`x = [30, 35, 40, 45, 50, 55, 60, 65, 70, 200]`

^^^ 一个异常值

MASTER 的做法

Step 1: 用鲁棒统计量做标准化

```
median = 52.5          # 中位数 (不受200影响)
mad = median(|x - 52.5|) = 15.0    # MAD (不受200影响)
robust_std = 15.0 * 1.4826 = 22.24 # 鲁棒标准差
```

标准化: $(x - \text{median}) / \text{robust_std}$

```
result = [-1.01, -0.79, -0.56, -0.34, -0.11,
          0.11, 0.34, 0.56, 0.79, 6.63]
```

^^^ 异常值变成了 6.63

Step 2: 硬裁剪到 $[-3, 3]$

```
result = [-1.01, -0.79, -0.56, -0.34, -0.11,
          0.11, 0.34, 0.56, 0.79, 3.00]
          ^^^ 被截断到 3.00
```

最终范围: 严格 $[-3, 3]$, 中心在 0 附近。

数据预处理对比

假设某天 RSI 因子在 10 只股票上的原始值:

$x = [30, 35, 40, 45, 50, 55, 60, 65, 70, 200]$

^^^ 一个异常值

L11 的做法

Step 1: 用MAD去极值

median = 52.5

mad = 15.0

upper = $52.5 + 5 * 15.0 = 127.5$ # 上界

lower = $52.5 - 5 * 15.0 = -22.5$ # 下界

$x_clipped = [30, 35, 40, 45, 50, 55, 60, 65, 70, 127.5]$

^^^^ 200被拉到127.5

Step 2: 用均值+标准差做Z-Score

mean = 57.75 # <-- 被127.5拉高了!

std = 28.37 # <-- 被127.5拉大了!

```
result = (x_clipped - 57.75) / 28.37
        = [-0.98, -0.80, -0.63, -0.45, -0.27,
           -0.10, 0.08, 0.26, 0.43, 2.46]
```

最终范围: 大约 $[-1, 2.5]$, 不固定。中心偏移了 (均值不在0) 。

数据预处理对比

Thinking: 核心差异在哪?

差异1: 中心化用什么

MASTER: $(x - \text{median})$ 中位数不受异常值影响, 52.5就是52.5

L11: $(x - \text{mean})$ 均值被200拉到了57.75, 偏离了真实中心

median如果要失真, 需要你改掉一半以上的数据才能让中位数失真。mean 只要一个极端值就会被拉偏。

差异2: 离散度用什么

MASTER: $/(MAD * 1.4826)$ MAD 也不受异常值影响

L11: $/std$ std 被异常值拉大, 导致正常值被压缩

MAD x 1.4826 这个系数的含义: 如果数据服从正态分布, MAD x 1.4826 恰好等于标准差。所以在正常数据上两者结果一样, 但在有异常值时 MAD 更稳定。

数据预处理对比

差异3：输出范围是否可控

MASTER: clip(-3, 3) --> 输出一定在 [-3, 3]

L11: 先去极值再Z-Score --> 输出范围取决于数据，每次不同

这对深度学习至关重要：神经网络的激活函数（ReLU、Sigmoid、Tanh）都有固定的有效输入范围：

激活函数	输入太大会怎样
Sigmoid	输入>5 时梯度趋近于 0（梯度消失）
Tanh	输入>3 时梯度趋近于 0
ReLU	不会饱和，但 BatchNorm/LayerNorm 需要输入范围稳定

数据预处理对比

Thinking: 为什么 L11 不用 MASTER 的方法?

因为 L11 用的是 XGBoost (树模型), 树模型有一个本质特性:

树模型只看排序, 不看数值大小

XGBoost 的分裂条件是 $x < \text{threshold}$, 不管 x 是 50 还是 50000, 只要排序没变, 模型输出完全一样。

所以: 输出范围固定不固定? => 无所谓, 树模型不关心

被拉偏了? => 影响不大, 排序基本没变

极端值? => 去掉就行, $\text{MAD} + 5\text{倍}$ 已经够了

但对 MASTER (Transformer / 深度学习) 来说, 数值的绝对大小直接影响梯度和训练稳定性, 所以要用更鲁棒的方法、严格控制输出范围。

CASE: MASTER截面预测

CASE：MASTER截面预测

TO DO：使用MASTER进行截面预测

使用论文作者提供的 预训练MASTER模型 和 配套数据集, 对CSI300股票进行截面预测, 验证论文Table 1报告的IC/ICIR/RankIC/RankICIR指标。

Dataset	Model	IC	ICIR	RankIC	RankICIR	AR	IR
CSI300	XGBoost	0.051 ± 0.001	0.37 ± 0.01	0.050 ± 0.001	0.36 ± 0.01	0.23 ± 0.03	1.9 ± 0.3
	LSTM	0.049 ± 0.001	0.41 ± 0.01	0.051 ± 0.002	0.41 ± 0.03	0.20 ± 0.04	2.0 ± 0.4
	GRU	0.052 ± 0.004	0.35 ± 0.04	0.052 ± 0.005	0.34 ± 0.04	0.19 ± 0.04	1.5 ± 0.3
	TCN	0.050 ± 0.002	0.33 ± 0.04	0.049 ± 0.002	0.31 ± 0.04	0.18 ± 0.05	1.4 ± 0.5
	Transformer	0.047 ± 0.007	0.39 ± 0.04	0.051 ± 0.002	0.42 ± 0.04	0.22 ± 0.06	2.0 ± 0.4
	GAT	0.054 ± 0.002	0.36 ± 0.02	0.041 ± 0.002	0.25 ± 0.02	0.19 ± 0.03	1.3 ± 0.3
	DTML	0.049 ± 0.006	0.33 ± 0.04	0.052 ± 0.005	0.33 ± 0.04	0.21 ± 0.03	1.7 ± 0.3
	MASTER	0.064* ± 0.006	0.42 ± 0.04	0.076* ± 0.005	0.49 ± 0.04	0.27 ± 0.05	2.4 ± 0.4
CSI800	XGBoost	0.040 ± 0.000	0.37 ± 0.01	0.047 ± 0.000	0.42 ± 0.01	0.08 ± 0.02	0.6 ± 0.2
	LSTM	0.028 ± 0.002	0.32 ± 0.02	0.039 ± 0.002	0.41 ± 0.03	0.09 ± 0.02	0.9 ± 0.2
	GRU	0.039 ± 0.002	0.36 ± 0.05	0.044 ± 0.003	0.39 ± 0.07	0.07 ± 0.04	0.6 ± 0.3
	TCN	0.038 ± 0.002	0.33 ± 0.04	0.045 ± 0.002	0.38 ± 0.05	0.05 ± 0.04	0.4 ± 0.3
	Transformer	0.040 ± 0.003	0.43 ± 0.03	0.048 ± 0.003	0.51 ± 0.05	0.13 ± 0.04	1.1 ± 0.3
	GAT	0.043 ± 0.002	0.39 ± 0.02	0.042 ± 0.002	0.35 ± 0.02	0.10 ± 0.04	0.7 ± 0.3
	DTML	0.039 ± 0.004	0.29 ± 0.03	0.053 ± 0.008	0.37 ± 0.06	0.16 ± 0.03	1.3 ± 0.2
	MASTER	0.052* ± 0.006	0.40 ± 0.06	0.066 ± 0.007	0.48 ± 0.06	0.28* ± 0.02	2.3* ± 0.3

CASE：MASTER截面预测

数据集：

项目	说明
数据来源	论文作者提供的opensource数据集
训练集	2008Q1 ~ 2020Q1
验证集	2020Q2
测试集	2020Q3 ~ 2022Q4 (619个交易日)
股票数量	433只 (CSI300成份股, 含历史调整)
样本总数	185,700条

CASE: MASTER截面预测

每个样本是一个 $(8, 222)$ 的矩阵:

维度说明: $(T=8, F=222)$

$T=8$: 回看窗口, 过去8个交易日的数据

$F=222$: 特征维度, 拆分如下

└─ $[0:158]$ Alpha158因子 (Qlib标准因子集)

└─ $[158:221]$ 63维市场信息 (CSI300/CSI500/CSI800指数特征)

└─ $[221]$ 标签 (未来5日收益率)

CASE：MASTER截面预测

模型超参数：

参数	CSI300	CSI800	说明
d_feat	158	158	Alpha158因子数
d_model	256	256	Transformer隐藏层维度
T_nhead (N1)	4	4	TAttention注意力头数
S_nhead (N2)	2	2	SAttention注意力头数
beta	5	2	Gate温度参数
dropout	0.5	0.5	Dropout率
模型参数量	775,041	775,041	约77.5万参数

beta选择的原因：

CSI300大盘股因子质量高, 用较大的beta(=5)做温和筛选;

CSI800含中小盘, 噪声大, 用较小的beta(=2)做强筛选。

CASE: MASTER截面预测

- 整体流程:

加载测试数据(pickle) --> 加载预训练模型(.pkl权重) --> 逐日截面预测 --> 计算IC指标 --> 可视化

CASE: MASTER截面预测

- MASTER模型前向传播

对应 master.py 中的 MASTER.forward()

```
def forward(self, x):  
    # x shape: (N, T, F) = (股票数, 8天, 222特征)  
    # 第1步: 分离股票因子和市场信息  
    src = x[:, :, :158]          # (N, 8, 158) Alpha158因子  
    gate_input = x[:, -1, 158:221]  # (N, 63) 最后一天的市场信息  
  
    # 第2步: Gate机制 - 用市场信息动态调整因子权重  
    # softmax(Linear(63->158) / beta) * 158  
    gate_weights = self.feature_gate(gate_input) # (N, 158)  
    src = src * gate_weights.unsqueeze(1)      # 逐因子加权  
    # 第3步: 线性映射 + 位置编码  
    # Linear(158 -> 256) + PositionalEncoding  
    # shape: (N, 8, 158) -> (N, 8, 256)
```

```
# 第4步: TAttention - 时序注意力 (股票内, 跨时间步)  
# 每只股票独立地在8个时间步间做self-attention  
# 捕捉: "RSI从30涨到65"这类时序模式
```

```
# 第5步: SAttention - 股票注意力 (跨股票, 同时间步)  
# 同一天的所有股票间做self-attention  
# 捕捉: 板块联动、龙头带动效应
```

```
# 第6步: TemporalAttention - 时序聚合  
# 将8个时间步的表示加权聚合为1个向量  
# 用最后一天作为query, 动态关注最重要的历史时刻
```

```
# 第7步: 线性解码  
# Linear(256 -> 1) 输出预测分数  
output = self.layers(src).squeeze(-1) # (N, 1) -> (N,)   
return output
```

2-MASTER截面预测.py

CASE: MASTER截面预测

- 逐日截面预测

预测采用 DailyBatchSampler, 每个batch是同一天的所有股票:

```
# DailyBatchSampler: 按交易日分批, 每批=一天的所有股票
sampler = DailyBatchSamplerRandom(dl_test, shuffle=False)
test_loader = DataLoader(dl_test, sampler=sampler, drop_last=False)

model.model.eval()
with torch.no_grad():
    for data in test_loader:
        # data shape: (1, N, T, F) -> squeeze -> (N, T, F)
        # N = 当天的股票数(约300只)
        data = torch.squeeze(data, dim=0)
        feature = data[:, :, 0:-1] # (N, 8, 221) 去掉最后一列标签
        label = data[:, -1, -1] # (N,) 最后一天的标签
```

```
pred = model.model(feature.float()) # (N,) 预测分数
```

```
# 当天截面IC: pred与label的Pearson/Spearman相关
```

```
daily_ic = pearson_corr(pred, label)
```

```
daily_ric = spearman_corr(pred, label)
```

- 每天约300只股票同时输入, SAttention能看到所有股票间的关系
- 预测输出是一个分数, 分数高的股票预期收益更高
- IC是逐日计算: 当天300只股票的预测排名 vs 实际收益排名的相关性

2-MASTER截面预测.py

CASE: MASTER截面预测

- IC指标计算

```
# 逐日IC的聚合
IC = mean(daily_ic_list)          # 日均IC
ICIR = mean(daily_ic_list) / std(daily_ic_list) # IC信息比率
RankIC = mean(daily_ric_list)      # 日均RankIC
RankICIR = mean(daily_ric_list) / std(daily_ric_list)
```

CASE：MASTER截面预测

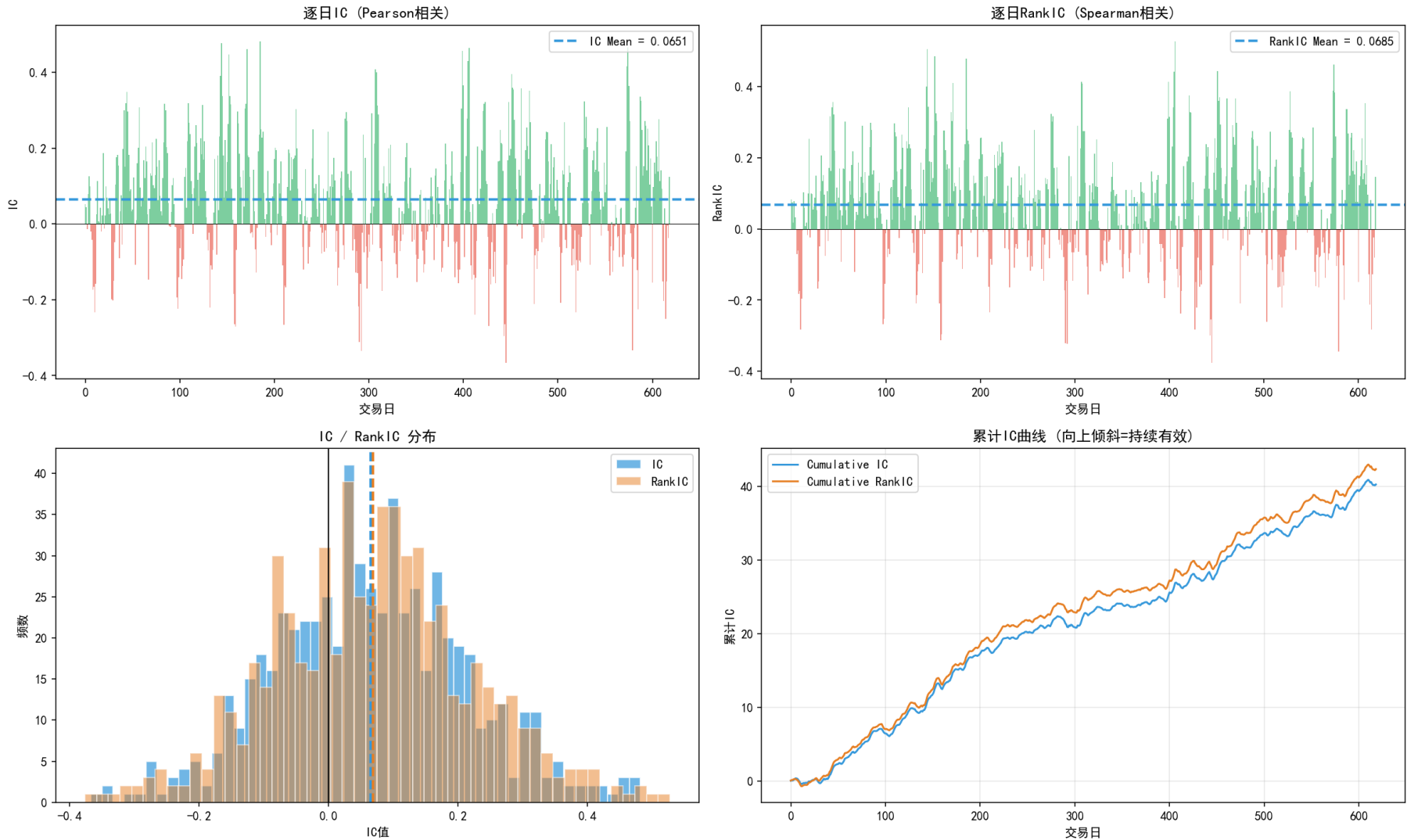
脚本运行成功! 结果与论文非常接近:

指标	本次结果	论文参考值	差异
IC	0.0651	0.0640	+0.0011
ICIR	0.4401	0.4200	+0.0201
RankIC	0.0685	0.0760	-0.0075
RankICIR	0.4490	0.4900	-0.0410

差异在合理范围内(论文是 5 次随机种子平均, 本次只用 seed=0)

CASE：MASTER截面预测

MASTER CSI300 截面预测评估



CASE：MASTER截面预测

结果指标解读

指标	含义	论文典型值	有效阈值
IC	预测值与真实收益的Pearson相关	0.05~0.08	> 0.05 有效
ICIR	IC均值/IC标准差，信号稳定性	0.4~0.7	> 0.3 稳定
RankIC	预测排名与真实排名的Spearman秩相关	0.08~0.12	> 0.08
RankICIR	RankIC均值/RankIC标准差	0.7~1.1	> 0.5

Thinking: 为什么RankIC通常比IC更高?

在实际投资中, 我们做的是选股排序(买前N只, 卖后N只), 而不是精确预测收益率的数值大小。
RankIC只关心排序正确性, 对异常值不敏感, 因此通常比IC更高, => 更直接地反映策略盈利能力。

CASE: XGBoost截面预测

从MASTER理论到实践

MASTER核心的贡献是IC/ICIR评估方法论，即用截面相关系数衡量预测质量。

这套方法论不依赖任何特定模型 => 我们用自己的数据和XGBoost来实践这套方法论, 同时也用论文预训练模型验证了MASTER本身的效果。

维度	MASTER 原版	我们的实践
相同	截面预测任务（预测多只股票相对排名）	截面预测任务
相同	IC/ICIR/RankIC/RankICIR 评估	IC/ICIR/RankIC/RankICIR 评估
相同	A股量价数据	A股量价数据
数据来源	Qlib.pkl（需下载）	我们的 MySQL（7177只A股）
模型	Transformer（双注意力+Gate）	XGBoost（L11已学过）
因子	158维 Alpha158 + 63维市场信息	52维 TA-Lib因子（L11 feature_engine）
基础设施	GPU + Qlib	CPU + MySQL

CASE: XGBoost截面预测

TO DO: 搭建A股截面预测实战

Step1, 加载50只A股大盘股: 从MySQL读取2023-2025年日K线(覆盖金融/消费/科技/制造/医药)

Step2, 计算52维技术因子: 调用L11的feature_engine(TA-Lib), 包括动量/波动率/RSI/MACD等

Step3, 单因子IC分析: 对每个因子计算截面IC/ICIR, 找出最有预测力的因子

Step4, 滚动XGBoost截面预测: 120天训练窗口, 每5天预测一次, 共120次

Step5, IC/ICIR评估: 计算与MASTER相同的评估指标, 对比论文值

CASE: XGBoost截面预测

运行结果

数据: 50只A股, 718交易日, 35,900行 x 52因子, 耗时2.7s

单因子IC Top 3:

momentum_20d IC=0.0528 ICIR=0.1874

bbands_position IC=0.0421 ICIR=0.1829

rsi_14 IC=0.0432 ICIR=0.1703

XGBoost截面预测 (120次预测, 耗时21.5s):

IC: 0.1877

ICIR: 0.7763

RankIC: 0.1434

RankICIR: 0.7450

IC>0占比: 77.5% (93/120)

实验结果：IC分析

对52个因子逐一计算截面IC, 找出最有预测力的因子:

因子	IC	ICIR	RankIC	含义
momentum_20d	0.0528	0.1874	0.0372	20日动量(ROC)
bbands_position	0.0421	0.1829	0.0265	布林带位置
rsi_bbands_cross	0.0417	0.1742	0.0205	RSI与布林交互因子
rsi_14	0.0432	0.1703	0.0294	14日RSI
ma_bull_score	0.0381	0.1671	0.0294	多头排列得分
macd_dif	0.0309	0.1612	0.0369	MACD差离值

实验结果：IC分析

各因子类别平均|ICIR|:

类别	平均ICIR	最佳因子	最佳因子ICIR
技术指标因子	0.136	bbands_position	0.183
交互因子	0.108	rsi_bbands_cross	0.174
均线与形态	0.101	ma_bull_score	0.167
动量因子	0.079	momentum_20d	0.187
价量因子	0.052	ret_5d	0.129
波动率因子	0.049	vol_change_20d	0.127

实验结果：IC分析

Thinking: 为什么技术指标因子的ICIR最高?

因为RSI、布林带、MACD等指标本身就是对价格动量和均值回归的综合度量

=> 相比原始的收益率或波动率, 它们已经做了一层特征工程。

=> 这与MASTER论文中Alpha158因子的设计思路一致

即在不同时间窗口下对量价数据进行多角度衍生。

实验结果

XGBoost截面预测 vs MASTER

用XGBoost在50只股票上做滚动截面预测(120天训练, 每5天预测), 120次有效预测的结果:

指标	我们(50只, XGBoost)	MASTER(300只, Transformer)	评估
IC	0.1877	0.05~0.08	达标
ICIR	0.7763	0.4~0.7	达标
RankIC	0.1434	0.08~0.12	达标
RankICIR	0.745	0.7~1.1	达标
IC>0占比	77.50%	--	信号可靠

实验结果（我们）

Thinking: 为什么我们的IC比MASTER高?

这不代表XGBoost比Transformer更强, 而是实验条件差异导致的:

1. 截面大小: 50只 vs 300只。截面越小, 排名越容易猜对, 但实际交易容量也小
2. 预测周期: 我们预测5日收益, MASTER预测1-2日。周期越长, 信号越明显
3. 时间窗口: 2023-2025年A股大盘股有明显的结构性趋势
4. 数据泄露风险: 50只股票的截面较小, 模型更容易记住而非预测

要公平对比, 需要在相同条件下(同样300只股票, 同样1日预测, 同样时间段)运行两个模型。MASTER的真正优势在于:

- Gate机制能根据市场环境动态选择因子
- S-Attention能捕捉板块联动效应
- T-Attention能利用多日时序模式

实验结果 (论文)

这里是论文中的CSI300数据集结果。 MASTER在6个指标中的5个上取得最佳。

Dataset	Model	IC	ICIR	RankIC	RankICIR	AR	IR
CSI300	XGBoost	0.051 ± 0.001	0.37 ± 0.01	0.050 ± 0.001	0.36 ± 0.01	0.23 ± 0.03	1.9 ± 0.3
	LSTM	0.049 ± 0.001	0.41 ± 0.01	0.051 ± 0.002	0.41 ± 0.03	0.20 ± 0.04	2.0 ± 0.4
	GRU	0.052 ± 0.004	0.35 ± 0.04	0.052 ± 0.005	0.34 ± 0.04	0.19 ± 0.04	1.5 ± 0.3
	TCN	0.050 ± 0.002	0.33 ± 0.04	0.049 ± 0.002	0.31 ± 0.04	0.18 ± 0.05	1.4 ± 0.5
	Transformer	0.047 ± 0.007	0.39 ± 0.04	0.051 ± 0.002	0.42 ± 0.04	0.22 ± 0.06	2.0 ± 0.4
	GAT	0.054 ± 0.002	0.36 ± 0.02	0.041 ± 0.002	0.25 ± 0.02	0.19 ± 0.03	1.3 ± 0.3
	DTML	0.049 ± 0.006	0.33 ± 0.04	0.052 ± 0.005	0.33 ± 0.04	0.21 ± 0.03	1.7 ± 0.3
	MASTER	0.064* ± 0.006	0.42 ± 0.04	0.076* ± 0.005	0.49 ± 0.04	0.27 ± 0.05	2.4 ± 0.4
CSI800	XGBoost	0.040 ± 0.000	0.37 ± 0.01	0.047 ± 0.000	0.42 ± 0.01	0.08 ± 0.02	0.6 ± 0.2
	LSTM	0.028 ± 0.002	0.32 ± 0.02	0.039 ± 0.002	0.41 ± 0.03	0.09 ± 0.02	0.9 ± 0.2
	GRU	0.039 ± 0.002	0.36 ± 0.05	0.044 ± 0.003	0.39 ± 0.07	0.07 ± 0.04	0.6 ± 0.3
	TCN	0.038 ± 0.002	0.33 ± 0.04	0.045 ± 0.002	0.38 ± 0.05	0.05 ± 0.04	0.4 ± 0.3
	Transformer	0.040 ± 0.003	0.43 ± 0.03	0.048 ± 0.003	0.51 ± 0.05	0.13 ± 0.04	1.1 ± 0.3
	GAT	0.043 ± 0.002	0.39 ± 0.02	0.042 ± 0.002	0.35 ± 0.02	0.10 ± 0.04	0.7 ± 0.3
	DTML	0.039 ± 0.004	0.29 ± 0.03	0.053 ± 0.008	0.37 ± 0.06	0.16 ± 0.03	1.3 ± 0.2
	MASTER	0.052* ± 0.006	0.40 ± 0.06	0.066 ± 0.007	0.48 ± 0.06	0.28* ± 0.02	2.3* ± 0.3

Summary

- 动量因子最有效:

momentum_20d(20日ROC)的ICIR最高, 说明A股大盘股在2023-2025年有较强的趋势延续性

- 技术指标因子类别领先:

RSI/布林带/MACD等经典指标的综合ICIR(0.136)最高, 因为它们是对原始量价的高阶提炼

- XGBoost+52因子产生了有效的IC信号:

IC=0.1877远超0.05的有效阈值, 说明截面预测方法论在我们的数据上确实可行

- 不能直接与MASTER对比:

截面大小(50 vs 300)、预测周期(5日 vs 1日)等条件不同, 高IC部分来自于更宽松的实验条件

打卡：股票截面预测



针对你的股票池（也可以是全量的大A股票），进行截面预测

- 设置标签，比如未来5天的归一化收益率
- Step1, 加载50只A股大盘股
- Step2, 计算52维技术因子
- Step3, 单因子IC分析：对每个因子计算截面IC/ICIR, 找出最有预测力的因子
- Step4, 滚动XGBoost截面预测：120天训练窗口, 每5天预测一次, 共120次
- Step5, IC/ICIR评估

你的自选股在未来5天的预期收益率如何？

前沿展望

量化AI的进化方向:

1. 华泰MASTER因子 (2024)

- 架构: GRU + 三层注意力(时间/因子/股票) + 门控机制
- 效果: 全市场选股, 累计超额收益8.4% (样本外2023)
- 意义: 国内券商落地版本, 验证了注意力机制在A股的有效性

2. AlphaNet系列 (华泰/天风, 2020-2024)

- 架构: 端到端因子挖掘, 神经网络自动组合基础因子
- 效果: 中证500增强组合超额收益超20%, $IR > 3$
- 意义: "让模型自己发现因子"比"人工设计因子"可能更有效

前沿展望

3. 逐笔成交Transformer模型 (学术前沿)

- 架构: 双层Transformer, 逐笔成交数据 → 日频信号
- 效果: 年化超额收益24.52%, IR = 4.04
- 意义: 数据粒度越细, 模型天花板越高

4. 遗传规划行业轮动 (学术前沿)

- 架构: 三目标遗传优化, 自动搜索行业轮动规则
- 效果: 年化超额收益19.20%
- 意义: 不是所有量化都要用深度学习, 进化算法也很强

技术演进路线

规则策略 → 统计模型 → 机器学习 → 深度学习 → 多模态AI

(均线/MACD) (线性回归) (XGBoost) (Transformer) (GPT+量价)

可解释性高 简单高效 平衡性好 性能天花板高 信息源最广

容量小 容量中等 容量中等 容量大 容量最大

核心不变: 好的数据 > 好的模型 > 好的算力



Thank You
Using data to solve problems